Homework 08 — STAT 432 (Fall 2026)

Name: Owen Park **NetID:** owenp3

Questions 1 and 2 are in Python (numpy 1.26, pandas 2.2, matplotlib) and have no randomness, so there is no seed. Neither question fits a classifier. Both only evaluate the score functions given in the assignment.

Question 1: Boundary and margin

Because $\|(0.8, 0.6)\|_2 = 1$, the distance from an observation to the boundary is just $|f(x)|$. An observation is inside the margin for its class when $yf(x) < 1$, on it when $yf(x) = 1$, and beyond it when $yf(x) > 1$. A negative $yf(x)$ means the rule puts it on the wrong side.

In [1]:
```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
pd.set_option("display.width", 130)

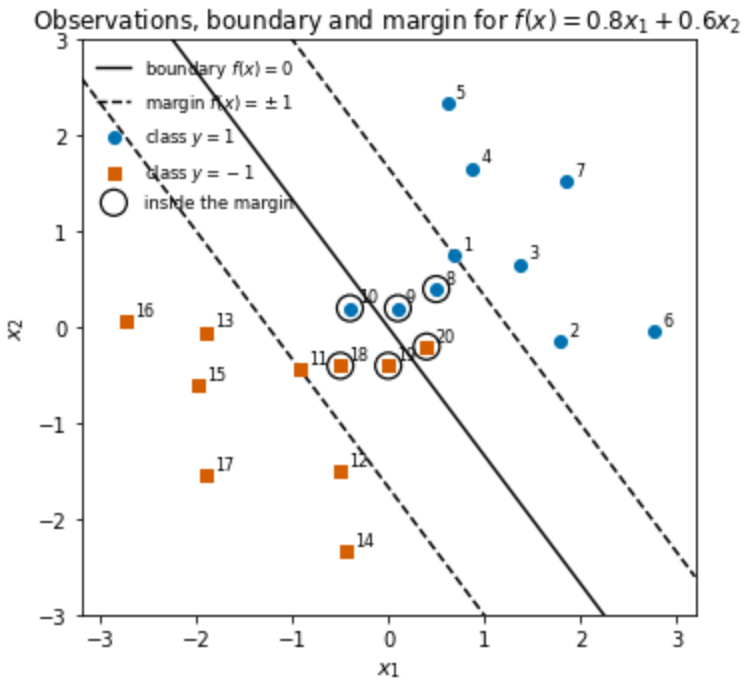
d = pd.read_csv("data/margin-geometry-20.csv")
w = np.array([0.8, 0.6])
d["f"] = d[["x1", "x2"]].to_numpy() @ w
d["distance"] = d["f"].abs() / np.linalg.norm(w)        # norm is 1, kept for clarity
d["yf"] = d["y"] * d["f"]
d["status"] = np.select([np.isclose(d["yf"], 1), d["yf"] < 1], ["on", "inside"], "beyond")
d["correct"] = np.where(d["yf"] > 0, "yes", "no")
print(d.round(2).to_string(index=False))
```

```
 observation    x1    x2  y     f  distance    yf status correct
           1  0.68  0.76  1  1.00      1.00  1.00     on     yes
           2  1.80 -0.15  1  1.35      1.35  1.35 beyond     yes
           3  1.38  0.66  1  1.50      1.50  1.50 beyond     yes
           4  0.88  1.66  1  1.70      1.70  1.70 beyond     yes
           5  0.62  2.34  1  1.90      1.90  1.90 beyond     yes
           6  2.78 -0.04  1  2.20      2.20  2.20 beyond     yes
           7  1.86  1.52  1  2.40      2.40  2.40 beyond     yes
           8  0.50  0.40  1  0.64      0.64  0.64 inside     yes
           9  0.10  0.20  1  0.20      0.20  0.20 inside     yes
          10 -0.40  0.20  1 -0.20      0.20 -0.20 inside      no
          11 -0.92 -0.44 -1 -1.00      1.00  1.00     on     yes
          12 -0.50 -1.50 -1 -1.30      1.30  1.30 beyond     yes
          13 -1.90 -0.05 -1 -1.55      1.55  1.55 beyond     yes
          14 -0.44 -2.33 -1 -1.75      1.75  1.75 beyond     yes
          15 -1.98 -0.61 -1 -1.95      1.95  1.95 beyond     yes
          16 -2.74  0.07 -1 -2.15      2.15  2.15 beyond     yes
          17 -1.90 -1.55 -1 -2.45      2.45  2.45 beyond     yes
          18 -0.50 -0.40 -1 -0.64      0.64  0.64 inside     yes
          19  0.00 -0.40 -1 -0.24      0.24  0.24 inside     yes
          20  0.40 -0.20 -1  0.20      0.20 -0.20 inside      no
```

(a) The table above has every observation's distance to the boundary, its functional margin $yf(x)$ and where it sits. Observations 1 and 11 have $yf(x)$ exactly 1, so they lie on the margin lines $f(x) = 1$ and $f(x) = -1$. The six with $yf(x) < 1$ are inside the margin, and observations 10 and 20 have negative $yf(x)$, which puts them past the boundary on the other class's side. Since the weight vector has length 1 the distance and $|yf(x)|$ are the same number, so the functional margin here is also the geometric distance with a sign that says whether the point is on the correct side.

In [2]:
```
blue, verm = "#0072B2", "#D55E00"
fig, ax = plt.subplots(figsize=(5.6, 5.0))
xx = np.linspace(-3.2, 3.2, 200)
for c, ls, lab in [(0, "-", "boundary $f(x)=0$"), (1, "--", "margin $f(x)=\\pm 1$"), (-1, "--", None)]:
    ax.plot(xx, (c - 0.8 * xx) / 0.6, color="black", ls=ls, lw=1.4, label=lab)
for cls, col, mk, lab in [(1, blue, "o", "class $y=1$"), (-1, verm, "s", "class $y=-1$")]:
    s = d[d["y"] == cls]
    ax.scatter(s["x1"], s["x2"], color=col, marker=mk, s=36, label=lab, zorder=3)
ins = d[d["status"] == "inside"]
ax.scatter(ins["x1"], ins["x2"], s=170, facecolors="none", edgecolors="black", lw=1.2, label="inside the margin", z
```

```
for _, r in d.iterrows():
    ax.annotate(int(r["observation"]), (r["x1"], r["x2"]), xytext=(4, 3), textcoords="offset points", fontsize=8)
ax.set_xlim(-3.2, 3.2); ax.set_ylim(-3, 3); ax.set_aspect("equal")
ax.set_xlabel("$x_1$", fontsize=11); ax.set_ylabel("$x_2$", fontsize=11)
ax.set_title("Observations, boundary and margin for $f(x)=0.8x_1+0.6x_2$", fontsize=12)
ax.tick_params(labelsize=10)
ax.legend(loc="upper left", frameon=False, fontsize=8.5)
fig.tight_layout(); plt.show()
```



(b) The plot shows the boundary as a solid line and the two margin lines dashed. Class $y = 1$ is blue circles, class $y = -1$ is vermilion squares, and the six observations inside the margin are circled in black.

In [3]:

```
print(d["status"].value_counts().reindex(["inside", "on", "beyond"]).to_string())
print("inside:", d.loc[d["status"] == "inside", "observation"].tolist())
print("misclassified:", d.loc[d["yf"] < 0, "observation"].tolist())
```

```
status
inside     6
on         2
beyond    12
inside: [8, 9, 10, 18, 19, 20]
misclassified: [10, 20]
```

(c) Six observations are inside the margin (8, 9, 10, 18, 19 and 20), two are on it (1 and 11), and the other twelve are beyond it. The rule misclassifies observations 10 and 20. Observation 10 is class 1 but has $f(x) = -0.20$, and observation 20 is class -1 but has $f(x) = 0.20$, so each sits 0.2 from the boundary on the wrong side. The other four points inside the margin are still classified correctly, they are just closer to the boundary than 1. That is the difference between being inside the margin and being misclassified, since only a negative $yf(x)$ makes the label wrong.

Question 2: Margin in four dimensions

The signed position of $\mathbf{x}$ along the unit direction $\boldsymbol{\beta}/\|\boldsymbol{\beta}\|$ is $\mathbf{x}^{\mathsf{T}}\boldsymbol{\beta}/\|\boldsymbol{\beta}\|$, and the boundary $\beta_0 + \mathbf{x}^{\mathsf{T}}\boldsymbol{\beta} = 0$ crosses that axis at $-\beta_0/\|\boldsymbol{\beta}\|$. Here $\|\boldsymbol{\beta}\| = \sqrt{1+4+4+16} = 5$, so the boundary is at $1/5 = 0.2$.

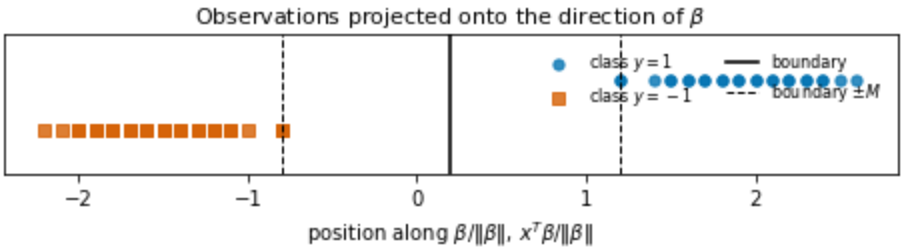
In [4]:

```
e = pd.read_csv("data/margin-geometry-4d.csv")
X = e[["x1", "x2", "x3", "x4"]].to_numpy(); y = e["y"].to_numpy()
beta0, beta = -1.0, np.array([1.0, 2.0, -2.0, 4.0])
nb_ = np.linalg.norm(beta)
proj = X @ beta / nb_                    # position on the beta axis
```

```
cut = -beta0 / nb_                          # boundary on that axis
f = beta0 + X @ beta
dist = y * f / nb_                          # signed distance to the boundary
print(f"||beta|| = {nb_:.0f}, boundary at {cut:.2f}, all correct: {(y * f > 0).all()}")
print(f"closest y=1 projection: {proj[y == 1].min():.2f}, closest y=-1 projection: {proj[y == -1].max():.2f}")
print(f"geometric margin M = {dist.min():.2f}, observations at M: {e.loc[np.isclose(dist, dist.min()), 'observation

fig, ax = plt.subplots(figsize=(6.5, 2.0))
rng_y = {1: 0.25, -1: -0.25}
for cls, col, mk, lab in [(1, blue, "o", "class $y=1$"), (-1, verm, "s", "class $y=-1$")]:
    ax.scatter(proj[y == cls], np.full((y == cls).sum(), rng_y[cls]), color=col, marker=mk, s=28, alpha=0.8, label=
ax.axvline(cut, color="black", lw=1.6, label="boundary")
ax.axvline(cut - 1, color="black", ls="--", lw=1.0); ax.axvline(cut + 1, color="black", ls="--", lw=1.0, label="bou
ax.set_yticks([]); ax.set_ylim(-0.7, 0.7)
ax.set_xlabel("position along $\\beta/\\|\\beta\\|$, $x^T\\beta/\\|\\beta\\|$", fontsize=10)
ax.set_title("Observations projected onto the direction of $\\beta$", fontsize=11)
ax.legend(loc="upper right", frameon=False, fontsize=8, ncol=2)
fig.tight_layout(); plt.show()
```

```
||beta|| = 5, boundary at 0.20, all correct: True
closest y=1 projection: 1.20, closest y=-1 projection: -0.80
geometric margin M = 1.00, observations at M: [4, 15, 33, 36]
```



(a) On the projection every class $y = 1$ observation lies to the right of the boundary at 0.2 and every class $y = -1$ observation to the left, so the rule separates the data. The closest $y = 1$ point sits at 1.2 and the closest $y = -1$ point at -0.8, both exactly 1 from the boundary, so the geometric margin is $M = 1$. Four observations (4, 15, 33 and 36) sit at that distance. The dashed lines at 0.2 ± 1 show that no point falls between them. Reading M off the axis works because the projection onto the unit vector $\boldsymbol{\beta}/\|\boldsymbol{\beta}\|$ keeps distances along that direction, and the other three directions are parallel to the boundary, so they don't change how far a point is from it.

(b) The smallest functional margin with the given coefficients is $\min_i y_i f(\mathbf{x}_i) = M\|\boldsymbol{\beta}\| = 1 \times 5 = 5$. Dividing β_0 and $\boldsymbol{\beta}$ by 5 keeps the same boundary and makes the smallest functional margin exactly 1, which gives

$$\boldsymbol{\beta} = (0.2,\, 0.4,\, -0.4,\, 0.8)^{\mathsf{T}}, \qquad \beta_0 = -0.2.$$

Under this normalization $\|\boldsymbol{\beta}\| = 1$, so $M = 1/\|\boldsymbol{\beta}\| = 1$ agrees with part (a). This is the scaling the SVM uses, where the margin is $1/\|\boldsymbol{\beta}\|$ once the closest points are pinned at $y_i f(\mathbf{x}_i) = 1$.

Question 3: Peer review of a Homework 7 skill

My assigned skill is `lda-qda-analysis` from lyndseybajgrowicz/stat432-fall2026, which is printed at the end of that student's Homework 7 report. I copied it without changes into `homework/week-08/peer-run/SKILL.md` and did not install it.

1. Using the skill. I gave a fresh Claude Code agent (Claude Opus 5.5) the path to the file and only the data information. The prompt was:

> Please follow the skill defined in this file exactly, and let it direct the whole analysis: `.../peer-run/SKILL.md`. Dataset: `saheart.csv` (the SAheart

> data from ElemStatLearn). Response: `chd`, the coronary heart disease indicator (two classes). Predictors: all nine other variables, used as numerical predictors, with `famhist` coded as 1 for `Absent` and 2 for `Present`.

I also told it which folder to write in and which software was installed, and asked it to list the places where it had to make a choice the skill did not specify. I did not need a steer message. The run finished in about 1 minute 45 seconds with 10 tool calls and about 62,000 tokens, and it wrote a one-page PDF report.

The agent picked seed 432, a stratified 75/25 split (346 training and 116 test observations), stratified 10-fold CV, and 31 candidate settings: plain LDA and QDA, PCA with 1 to 9 components followed by LDA or QDA, and LDA/QDA probability blends with weights 0 to 1 in steps of 0.1. It selected PCA(7)+LDA with a mean CV error of 0.254 (SE 0.025), against 0.266 (SE 0.028) for plain LDA and 0.304 for plain QDA. The reserved test error was 0.267 (31 of 116), with 20 of the 40 cases detected. For comparison, the author's own run used a 25% test split, 5-fold CV and 41 settings, and selected PCA(4)+QDA with CV error 0.246 and test error 0.302.

2. Peer review.

Overall assessment. This is a short, clear skill (about 2.8 KB) that gets the leakage rules right. It reserves the test set before modeling, fits all preprocessing inside each fold, uses the same folds for every candidate, and says not to quietly swap in another model when a fit fails. When I gave it to an agent with only the data information, it ran start to finish without asking anything and produced a one-page report with a table, a figure and the CV and test errors. My three suggestions are about the choices the skill leaves open, the selection rule, and what the report says about the errors.

Item 1: Many design choices are left to the agent, so two runs can disagree.

- *Observed.* The skill doesn't give the test split proportion, the number of folds, the PCA or blend grids, or whether to standardize before PCA. My agent listed these as choices it had to make itself. It used 10-fold CV, standardized before PCA, and compared 31 settings, and it selected PCA(7)+LDA. The author's run used 5-fold CV and 41 settings and selected PCA(4)+QDA.
- *Why it matters.* The same skill on the same data picked a different classifier family. So the selected model depends on choices the reader of the report never sees, and the skill doesn't give reproducible results across users.
- *Recommendation.* Set defaults in the skill, for example a stratified 25% test set, 5-fold CV, PCA dimensions 1 to p fit on standardized predictors inside each fold, and blend weights from 0 to 1 in steps of 0.1. Also require the report to list every choice the agent made that the skill did not fix.

Item 2: "Lowest mean CV error" picks winners that are within the noise.

- *Observed.* In my run, PCA(7)+LDA had a mean CV error of 0.254 and plain LDA had 0.266, a difference of about 0.012 when the fold SE was about 0.025. In the author's run,

the selected PCA(4)+QDA had CV error 0.246 but test error 0.302 (35 of 116), so the CV number was optimistic.

- *Why it matters.* When the top candidates are within one SE of each other, the choice between them is mostly fold noise, and a more complex model can win just by chance. The selected model's CV error is also biased low because the same folds chose it.
- *Recommendation.* Use a one-standard-error rule: among candidates within one SE of the best, choose the simplest (for example LDA, then QDA, then PCA models with fewer components, then blends). Ask for the SE next to every mean CV error in the table, and have the report say that the selected model's CV error is optimistic.

Item 3: Misclassification rate alone hides how the model does on the cases.

- *Observed.* The skill only asks for the overall misclassification rate at the default cutoff. In my run the selected model got 65 of 76 non-cases right but only 20 of the 40 cases (50% sensitivity), and the agent mentioned this only because it chose to on its own.
- *Why it matters.* The classes are unbalanced (about one third cases), so a model can have a decent overall error while missing half of the people who have the disease. For a medical response like this, that is the most important number for a reader.
- *Recommendation.* Require the test confusion matrix and the error rate within each class in the report, and state the cutoff used. The skill could also ask the agent to note when one class's error is much higher than the overall error.

Other notes. I did not stop the run early; it took about 1 minute 45 seconds. The skill also doesn't say where to save the report or in what format, so it would help to add one line about the output file.

3. GitHub Issue. The same review is posted at https://github.com/lyndseybajgrowicz/stat432-fall2026/issues/3.